Software Requirements Specification (SRS)
JustGo Multi-Tenant SaaS REST API Platform

Document Control

Document Information	Value
Title	Software Requirements Specification for JustGo Multi-Tenant SaaS REST API Platform
Date	April 25, 2026
Status	First Draft - Reverse-engineered from available codebase
Version	0.1
Prepared for	JustGo Technologies Limited
Prepared by	Software Architecture Review
Reference	SRS_JustGoAPI_First_Draft_v0.1

Disclaimer: This first draft is based on static source-code inspection of the uploaded JustGoAPI.API codebase and the provided Annex-A SRS sample. Business rules, user journeys, data ownership, tenant isolation rules, operational SLAs, and missing module source code require validation with JustGo stakeholders. Items marked TBD must be confirmed before baseline approval.

Table of Contents

1. Introduction

1.1. Purpose

1.2. Document Conventions

1.3. Intended Audience

1.4. Project Scope

1.5. References

2. Overall Description

2.1. Product Perspective

2.2. Product Functions

2.3. User Classes, Characteristics, and Needs

2.4. Operating Environment

2.5. Design and Implementation Constraints

2.6. User Documentation

2.7. Assumptions and Dependencies

3. System Features and Functional Requirements

4. External Interface Requirements

5. Non-Functional Requirements

6. Other Requirements

6.4. Appendix A: Analysis Models

6.5. Appendix B: Issues List

Glossary

Term	Definition
ABAC	Attribute-Based Access Control
API	Application Programming Interface
DTO	Data Transfer Object
JWT	JSON Web Token
MFA	Multi-Factor Authentication
RBAC	Role-Based Access Control
SaaS	Software as a Service
SLA	Service Level Agreement
SRS	Software Requirements Specification
TBD	To Be Determined

1. Introduction

1.1. Purpose

This Software Requirements Specification describes the JustGo multi-tenant SaaS REST API platform represented by the JustGoAPI.API host and its modular API codebase. The document captures the system purpose, product perspective, functional requirements, external interfaces, quality attributes, and known gaps discovered during initial static analysis.

The SRS is intended to become the baseline documentation for further architecture analysis, QA planning, business validation, API documentation, onboarding of engineers, and future modernization or product roadmap work.

1.2. Document Conventions

Term	Description
SHALL	Mandatory requirement for the current API platform.
SHOULD	Important requirement that should be implemented or improved where not already present.
MAY	Optional or future requirement.
TBD	Information unavailable from code and requiring stakeholder confirmation.
Note	Clarification or implementation observation.
Requirement Number	**Description**
FR-XX	Functional requirement
NFR-XX	Non-functional requirement
IR-XX	Interface requirement
DR-XX	Data requirement
SR-XX	Security requirement

1.3. Intended Audience

Stakeholder	Role
Product owners	Validate business scope and module behavior.
Engineering team	Use requirements for development, refactoring, test coverage, and onboarding.
QA team	Derive test cases, regression suites, and acceptance criteria.
DevOps/SRE	Plan deployment, monitoring, scaling, backup, and operational support.
Security/compliance reviewers	Validate authentication, authorization, audit, privacy, and data protection controls.
Integration partners	Understand API boundaries and integration expectations.

1.4. Project Scope

The JustGoAPI platform is a modular REST API backend for a multi-tenant SaaS system serving sports clubs, organizations, administrators, members, and related operational workflows.

In Scope

- Authentication, account, MFA, token, tenant, and user APIs
- Common file attachment, notes, lookup, cache invalidation, and UI permission APIs
- Asset management APIs for asset registers, categories, ownership, leasing, licensing, credentials, workflows, audit, and reports
- Booking APIs for catalog, classes, sessions, attendees, terms, pricing discounts, transfer checks, and profile bookings
- Credential APIs for member credential summary, credential metadata, and credential categories
- Field management APIs for dynamic entity extensions, schemas, data, attachments, and user weblet preferences
- Platform-level API versioning, Swagger/OpenAPI, JWT authentication, CORS, response compression, SignalR progress hub, Serilog logging, and exception handling
- Referenced but source-missing modules: Finance, MemberProfile, Membership, Organisation, MobileApps, and Result. These are visible as project references and runtime assemblies but require source-level validation.

Out of Scope for this first draft

- Detailed UI requirements outside REST API behavior
- Confirmed business process diagrams for modules whose source code was not available in the extracted source tree
- Production infrastructure sizing, RTO/RPO, and contractual SLA values until confirmed
- Payment provider business rules beyond references observable from dependencies and API names
- Legal and compliance commitments beyond recommended requirements

1.5. References

Reference	Description
Annex-A-Detailed-Software-Requirements-Specification-SRS.pdf	Sample SRS format supplied by the user.
JustGoAPI.API.zip	Uploaded API codebase archive.
JustGoAPI.API/Program.cs	Main ASP.NET Core API host configuration.
Controller source files	Primary source for discovered REST API capabilities.
Project files	Source for target framework, dependencies, and module references.

2. Overall Description

2.1. Product Perspective

JustGoAPI is an ASP.NET Core REST API host targeting .NET 9. The API host composes multiple business modules using dependency injection. It exposes versioned HTTP APIs, Swagger/OpenAPI documentation, JWT authentication, custom authorization middleware, centralized exception handling, CORS, static files, response compression, Serilog logging, and a SignalR progress-reporting hub.

The solution follows a modular architecture with API, Application, Domain, Infrastructure, and Test projects for available modules. Available source modules include Auth, Asset Management, Booking, Credential, and Field Management. Additional modules are referenced by the API host but were not present as source code in the extracted tree; their compiled assemblies appear in build output.

Context diagram in text form: Client applications and administrators interact with the JustGo REST API. The API authenticates users, resolves tenant context, executes module services, accesses SQL Server and file storage, integrates with external services such as email/SMS/payment/blob storage where configured, and emits logs/audit events.

2.2. Product Functions

1) User authentication, token refresh, password reset, password change, hash/encryption helper endpoints, user creation and profile lookup

2) MFA setup, verification, backup codes, OTP delivery, mandatory MFA enforcement, and admin MFA management

3) Tenant CRUD and domain-based tenant lookup

4) ABAC policy evaluation and UI permission lookup

5) File upload/download and entity attachment management

6) Entity notes management

7) Reference data lookup for countries, counties, genders, regions, and club types

8) Asset type, category, register, credential, lease, license, ownership transfer, workflow, audit, report, status, and club-related APIs

9) Booking catalog, classes, occurrences, attendees, pricing chart discounts, transfer checks, class terms, profile class/course bookings, and QR booking link APIs

10) Credential summary and metadata APIs

11) Dynamic field/entity extension schema, UI schema, data persistence, attachments, and weblet preference APIs

12) Cross-cutting API versioning, Swagger, JWT, CORS, logging, exception handling, response compression, and real-time progress notifications

2.3. User Classes, Characteristics, and Needs

User Class	Characteristics and Needs
Platform administrators	Manage tenants, settings, users, policies, MFA, cache, files, and support activities.
Club/organization administrators	Manage clubs, organization context, members, assets, booking offerings, credentials, and reporting.
Members/end users	Authenticate, manage profile-linked bookings/assets/credentials, access member-specific class/course data, and interact with account services.
Finance/payment operators	Use referenced finance/payment functionality; exact workflows TBD from missing source module.
Integration clients/mobile apps	Consume REST APIs using JWT and tenant-aware context; mobile-specific APIs are referenced but source was unavailable.
Support engineers	Use logs, audit trails, attachment data, and API diagnostics to troubleshoot production issues.

2.4. Operating Environment

- Server: ASP.NET Core/.NET 9 runtime hosted in a secure environment.
- Database: SQL Server access is indicated by Microsoft.Data.SqlClient, EF Core SQL Server, Dapper, and stored-query patterns.
- File storage: Azure Blob related infrastructure is referenced; local/static files are also enabled.
- API clients: web applications, mobile applications, admin portals, and partner integrations over HTTPS.
- Real-time channel: SignalR hub exposed at /progressReportHub.
- Documentation: Swagger/OpenAPI enabled through application configuration.

2.5. Design and Implementation Constraints

- The API SHALL remain tenant-aware and protect tenant data boundaries.
- The API SHALL use versioned endpoints to preserve backward compatibility.
- The API SHOULD maintain modular boundaries between API, Application, Domain, and Infrastructure layers.
- The API SHALL use JWT authentication and custom authorization middleware before controller execution.
- The API SHOULD avoid placing security-sensitive utility endpoints in production unless explicitly required and protected.
- Source for several referenced modules is missing in the supplied archive; requirements for those modules are provisional.

2.6. User Documentation

- API onboarding guide
- Swagger/OpenAPI reference
- Tenant setup and operations guide
- Administrator guide for users, MFA, cache, files, notes, and policies
- Module guides for Asset Management, Booking, Credential, Field Management, Finance, Member Profile, Membership, Organisation, Mobile Apps, and Result
- Operational runbooks for deployment, monitoring, backup, incident response, and troubleshooting

2.7. Assumptions and Dependencies

Assumptions

- Tenant isolation is based on domain and/or tenant identifiers resolved by authentication and tenant services.
- Authentication tokens carry claims required by module services and UI permissions.
- SQL Server is the primary operational datastore.
- Azure Blob or equivalent storage is used for file persistence in production.
- Missing source modules are part of the production product although not available in the extracted source tree.

Dependencies

- SQL Server connectivity and stored procedures/queries
- External payment providers where finance/booking flows require payment
- Email/SMS/OTP providers for MFA and password flows
- Azure Blob or configured file storage
- Swagger/OpenAPI consumers and API gateways, if used
- Tenant domain/DNS configuration

3. System Features and Functional Requirements

This section lists first-draft functional requirements derived from static code inspection. Final wording and priority SHALL be validated with business stakeholders.

3.1. Platform Authentication, Accounts, and User Management

ID	Requirement
FR-001	The system SHALL authenticate users through a REST endpoint and issue access tokens for authorized API access.
FR-002	The system SHALL support authentication by existing token where configured.
FR-003	The system SHALL support refresh token operations.
FR-004	The system SHALL allow users to initiate forgot-password flows.
FR-005	The system SHALL allow authenticated or verified users to change passwords.
FR-006	The system SHALL expose user lookup by login identifier and by user GUID.
FR-007	The system SHALL allow authorized administrators or system processes to create users.
FR-008	The system SHALL allow authorized administrators or system processes to update users.
FR-009	The system SHALL enforce role, group, policy, or claim-based authorization before protected operations.
FR-010	The system SHALL return consistent API responses for successful and failed account operations.

3.2. Multi-Factor Authentication

ID	Requirement
FR-011	The system SHALL provide MFA setup for supported authenticator methods.
FR-012	The system SHALL verify MFA codes submitted by users.
FR-013	The system SHALL support removal of authenticator configuration for authorized users.
FR-014	The system SHALL send OTP codes where the configured MFA channel requires OTP delivery.
FR-015	The system SHALL validate whether a user is subject to mandatory MFA.
FR-016	The system SHALL allow authorized administrators to enable or disable MFA for users.
FR-017	The system SHALL allow users to enable or disable their own MFA where policy permits.
FR-018	The system SHALL provide backup code retrieval or management for users.
FR-019	The system SHALL log MFA actions for audit and troubleshooting.
FR-020	The system SHALL expose supported country phone codes for phone-based MFA configuration.

3.3. Tenant and System Settings Management

ID	Requirement
FR-021	The system SHALL provide tenant listing and tenant retrieval by numeric identifier.
FR-022	The system SHALL provide tenant retrieval by GUID.
FR-023	The system SHALL provide tenant lookup by domain and tenant GUID lookup by domain.
FR-024	The system SHALL allow authorized administrators to create tenants.
FR-025	The system SHALL allow authorized administrators to update tenants.
FR-026	The system SHALL allow authorized administrators to delete tenants where business rules permit.
FR-027	The system SHALL support system settings persistence through an authorized endpoint.
FR-028	The system SHALL ensure tenant operations are restricted to authorized platform-level users.

3.4. Authorization Policies and UI Permissions

ID	Requirement
FR-029	The system SHALL evaluate ABAC policies for named policy and action attributes.
FR-030	The system SHALL return policy definitions or policy evaluation metadata by policy name.
FR-031	The system SHALL provide field-level permission metadata for policy-driven UI rendering.
FR-032	The system SHALL provide UI permission lookup by policy name.
FR-033	The system SHALL provide UI permission lookup by policy name and request parameters.
FR-034	The system SHALL ensure authorization decisions are traceable and testable.

3.5. Files, Attachments, Notes, and Lookup Data

ID	Requirement
FR-035	The system SHALL support file upload through multipart or equivalent API requests.
FR-036	The system SHALL support Base64 file upload where required by clients.

FR-037	The system SHALL support temporary and permanent file download flows.
FR-038	The system SHALL support public download for explicitly public files only.
FR-039	The system SHALL list entity attachments by entity type, entity identifier, and module.
FR-040	The system SHALL support offset and keyset pagination for attachment lists.
FR-041	The system SHALL allow authorized users to add entity attachments.
FR-042	The system SHALL allow authorized users to delete entity attachments.
FR-043	The system SHALL allow authorized users to download entity attachments.
FR-044	The system SHALL allow authorized users to list, add, update, and delete notes for module entities.
FR-045	The system SHALL provide lookup values for countries, counties, gender, regions, and club types.

3.6. Cache Invalidation

ID	Requirement
FR-046	The system SHALL allow authorized users or services to invalidate cache entries by key.
FR-047	The system SHALL allow authorized users or services to invalidate cache entries by tag.
FR-048	The system SHALL allow authorized users or services to invalidate cache entries by pattern.
FR-049	The system SHOULD clearly separate public and protected cache invalidation endpoints.
FR-050	The system SHALL audit cache invalidation requests.

3.7. Asset Management

ID	Requirement
FR-051	The system SHALL list asset categories for a given asset type.
FR-052	The system SHALL list asset types and retrieve asset type details.
FR-053	The system SHALL allow authorized users to create, save, list, view, delete, reinstate, and submit asset registers.
FR-054	The system SHALL support asset register status changes and maintain status history.
FR-055	The system SHALL provide asset register duplicate checking.
FR-056	The system SHALL provide notifications and journey completion steps for asset registers.
FR-057	The system SHALL support asset credential creation, editing, status change, credential product lookup, and permission lookup.
FR-058	The system SHALL support asset lease creation, editing, list views, activity logs, lease history, status changes, details, additional fee lookup, and owner approval metadata.
FR-059	The system SHALL support asset license metadata, creation, editing, cancellation, additional fees, deletion, upgrade metadata, cart validation, status changes, purchasable items, definitions, and permission lookup.
FR-060	The system SHALL support asset ownership transfer creation, history, permission lookup, details, activity logs, status changes, and owner approval metadata.
FR-061	The system SHALL provide asset metadata including credentials, tags, statuses, lease statuses, clubs, members, action reasons, asset details, additional forms, and additional fees.
FR-062	The system SHALL provide asset audit listing.

FR-063	The system SHALL provide asset report retrieval and download.
FR-064	The system SHALL support workflow submission for asset management processes.

3.8. Booking and Class Management

ID	Requirement
FR-065	The system SHALL provide booking catalog endpoints for disciplines, age groups, descriptions, filter metadata, and basic club details.
FR-066	The system SHALL list booking classes and return class details.
FR-067	The system SHALL return occurrence details for booking classes.
FR-068	The system SHALL return attendee and attendee payment information.
FR-069	The system SHALL return payment form information for attendees.
FR-070	The system SHALL return class group details and primary club GUID.
FR-071	The system SHALL allow authorized users to update invited users for class bookings.
FR-072	The system SHALL provide attendee list and session resolution operations.
FR-073	The system SHALL manage pricing chart discounts including create, update, status update, delete, list, and dropdown data.
FR-074	The system SHALL check member plan status for booking transfer requests.
FR-075	The system SHALL provide attendee occurrence calendar views and pro-rata calculations.
FR-076	The system SHALL provide term lookup data and holiday removal for class terms.
FR-077	The system SHALL provide profile class booking list and past class details.
FR-078	The system SHALL provide profile course booking details, cancellation, and booking QR links.

3.9. Credential Management

ID	Requirement
FR-079	The system SHALL provide credential summary for a member by user GUID.
FR-080	The system SHALL list member credentials using query/filter criteria.
FR-081	The system SHALL provide credential categories.
FR-082	The system SHALL provide credential metadata for a selected credential identifier.

3.10. Field Management and Entity Extensions

ID	Requirement
FR-083	The system SHALL provide UI tab items for owner, entity, and extension context.
FR-084	The system SHALL provide UI tab organizations available to a user.
FR-085	The system SHALL provide dynamic UI schema for entity extension forms.
FR-086	The system SHALL provide arena-specific UI schema where configured.
FR-087	The system SHALL provide backend schema for extension forms.
FR-088	The system SHALL create or update extension schema by tab item identifier.

FR-089	The system SHALL delete configured extension forms where authorized.
FR-090	The system SHALL list field-level attachments for dynamic forms.
FR-091	The system SHALL upload fieldset attachments.
FR-092	The system SHALL retrieve and persist dynamic entity extension data.
FR-093	The system SHALL retrieve and persist form data by extension and entity identifiers.
FR-094	The system SHALL manage user weblet preferences by user and preference type.

3.11. Referenced Modules Requiring Source Validation

ID	Requirement
FR-095	The system SHALL document Finance module APIs after source code becomes available.
FR-096	The system SHALL document Member Profile module APIs after source code becomes available.
FR-097	The system SHALL document Membership module APIs after source code becomes available.
FR-098	The system SHALL document Organisation module APIs after source code becomes available.
FR-099	The system SHALL document Mobile Apps module APIs after source code becomes available.
FR-100	The system SHALL document Result module APIs after source code becomes available.

4. External Interface Requirements

4.1. User Interfaces

The uploaded codebase represents backend REST APIs. User interfaces are external clients such as web admin portals, member portals, mobile apps, and partner systems. UI requirements are therefore expressed as API support requirements.

Interface	Description
Admin portal	Tenant, user, MFA, policy, cache, file, note, asset, booking, credential, and field management functions.
Member portal	Account access, member-specific bookings, credentials, assets, course/class data, and attachments.
Mobile apps	Mobile APIs are referenced by project configuration but source validation is TBD.
Partner/integration clients	REST API access using HTTPS, JSON, JWT, tenant context, and versioned endpoints.

4.2. Hardware Interfaces

No direct hardware interfaces were identified in source. The system depends on server, storage, database, network, and backup infrastructure.

4.3. Software Interfaces

Software Interface	Description
SQL Server	Primary data access through EF Core SQL Server, Microsoft.Data.SqlClient, System.Data.SqlClient, and Dapper.
Azure Blob/File Storage	Referenced by file system manager and file APIs.
JWT provider	Used for bearer authentication.
Email/SMS/OTP provider	Required for MFA, OTP, and password workflows; exact provider TBD.
Payment provider	Adyen assembly is present in build output; exact business usage TBD from missing source modules.

Swagger/OpenAPI	API documentation and testing interface.
SignalR	Real-time progress reporting hub at /progressReportHub.
Serilog sinks	Console, event, and exception logging sinks.

4.4. Communications Interfaces

ID	Requirement
IR-001	The system SHALL expose RESTful APIs over HTTPS.
IR-002	The system SHALL use JSON as the primary request and response format.
IR-003	The system SHALL expose Swagger/OpenAPI documentation for discoverable APIs.
IR-004	The system SHALL support API versioning for versioned controllers.
IR-005	The system SHALL support SignalR communications for progress reports.
IR-006	The system SHALL use secure JWT bearer authentication for protected APIs.
IR-007	The system SHALL apply configured CORS policies to browser-based clients.

5. Non-Functional Requirements

5.1. Performance Requirements

ID	Requirement
NFR-001	The system SHALL respond to standard API requests within 3 seconds under normal load, excluding external provider latency.
NFR-002	The system SHALL paginate large result sets using offset or keyset pagination where supported.
NFR-003	The system SHALL use response compression over HTTPS.
NFR-004	The system SHOULD cache frequently accessed reference data and invalidate cache explicitly when required.
NFR-005	The system SHOULD define benchmark targets for asset lists, booking lists, file operations, and authentication flows.

5.2. Security Requirements

ID	Requirement
NFR-006	The system SHALL require HTTPS in production.
NFR-007	The system SHALL protect APIs with JWT authentication except intentionally public endpoints.
NFR-008	The system SHALL enforce authorization using policies, roles, claims, ABAC, and/or UI permission rules.
NFR-009	The system SHALL support MFA for users and administrators according to configurable policy.
NFR-010	The system SHALL protect file download, attachment, cache, and administrative endpoints from unauthorized access.
NFR-011	The system SHALL avoid exposing hash/encrypt/decrypt helper endpoints in production unless strictly required and strongly restricted.
NFR-012	The system SHALL log security-relevant authentication, authorization, MFA, attachment, and administrative events.
NFR-013	The system SHALL prevent cross-tenant access to data and configuration.

5.3. Reliability and Availability

ID	Requirement
NFR-014	The system SHALL provide centralized exception handling

	and consistent problem responses.
NFR-015	The system SHALL log application requests, events, and exceptions.
NFR-016	The system SHOULD define availability targets for production and non-production environments.
NFR-017	The system SHOULD implement retry/circuit-breaker patterns for external dependencies.
NFR-018	The system SHOULD define backup, restore, RTO, and RPO targets for SQL Server and file storage.

5.4. Usability and Accessibility

ID	Requirement
NFR-019	The API SHALL provide consistent naming, response structure, and error formats for client usability.
NFR-020	The API SHALL expose Swagger/OpenAPI documentation for discoverability.
NFR-021	The API SHOULD provide clear validation messages for invalid request payloads.
NFR-022	Client applications SHOULD meet WCAG accessibility targets; backend APIs SHALL provide data necessary for accessible UI rendering where applicable.

5.5. Maintainability and Portability

ID	Requirement
NFR-023	The system SHALL preserve modular boundaries and dependency injection registration per module.
NFR-024	The system SHALL maintain automated tests for business-critical commands, queries, controllers, and validation rules.
NFR-025	The system SHOULD document module ownership, route maps, data models, and workflows.
NFR-026	The system SHOULD keep configuration outside code and use environment-specific settings.
NFR-027	The system SHOULD support containerized or repeatable deployment.

5.6. Legal and Compliance Requirements

ID	Requirement
NFR-028	The system SHALL define data retention and privacy rules for user data, files, MFA logs, notes, audit events, and tenant data.
NFR-029	The system SHALL support auditability of administrative and security-sensitive operations.
NFR-030	The system SHALL respect applicable local privacy, consumer, payment, and sports organization regulations.
NFR-031	The system SHALL document third-party component licenses and provider agreements.

5.7. Operational Requirements

ID	Requirement
NFR-032	The system SHALL expose health, logging, monitoring, and diagnostic capabilities suitable for production operations.
NFR-033	The system SHALL document deployment steps, configuration keys, secrets, and infrastructure dependencies.
NFR-034	The system SHALL maintain log retention and secure access to logs.
NFR-035	The system SHOULD provide dashboards or alerts for authentication failures, MFA failures, API errors, slow queries, and storage failures.
NFR-036	The system SHALL document support procedures for tenant setup, user lockout, MFA reset, file recovery,

	booking/payment incidents, and asset workflow issues.

6. Other Requirements

6.1. Data Migration

- The system SHALL define migration procedures for tenant, user, membership, asset, booking, credential, attachment, and extension data where applicable.
- The system SHALL validate migrated data for completeness, consistency, tenant ownership, and referential integrity.
- The system SHALL provide rollback and reconciliation procedures for migration defects.

6.2. Internationalization Requirements

- The API SHOULD support localization-ready lookup data and messages where clients require multiple languages.
- Date, time, currency, phone number, and address formats SHOULD be normalized and documented.
- Tenant-specific localization and terminology SHOULD be supported where product configuration requires it.

6.3. Training Requirements

- Developer onboarding for modular architecture, API conventions, authentication, authorization, logging, and testing.
- Administrator training for tenant management, user support, MFA support, files, notes, cache, and module operations.
- QA training for deriving API tests from this SRS and Swagger route maps.
- Operations training for deployment, monitoring, backup, incident response, and provider troubleshooting.

6.4. Appendix A: Analysis Models

A.1 Context Diagram

Element	Description
Client Applications	Web portal, admin portal, member portal, mobile apps, partner systems.
JustGoAPI Host	ASP.NET Core API host, versioning, Swagger, middleware, authentication, logging, SignalR.
Business Modules	Auth, Asset Management, Booking, Credential, Field Management, and referenced modules.
Data Stores	SQL Server and file/blob storage.
External Services	Email/SMS/OTP, payment provider, logging/monitoring infrastructure.

A.2 Module Diagram

Module	Observed Responsibility
JustGoAPI.API	Composes controllers and modules, configures middleware and hosted API services.
AuthModule	Accounts, users, tenants, MFA, policies, UI permissions, files, notes, lookups, cache.
AssetManagementModule	Asset registers, categories, credentials, leases, licenses, ownership transfers, workflows, audits, reports.
BookingModule	Catalog, classes, occurrences, attendees, payments, pricing discounts, transfer checks, terms, profile bookings.
CredentialModule	Member credential summary, member credential list, categories, metadata.
FieldManagementModule	Dynamic entity extension schema, UI schema, form data, attachments, preferences.
Referenced modules	Finance, MemberProfile, Membership, Organisation, MobileApps, Result - source validation required.

A.3 API Surface Summary

Controller Area	Discovered HTTP Actions
AssetManagement / AssetAudit	1
AssetManagement / AssetCategories	1

AssetManagement / AssetCheckout	2
AssetManagement / AssetCredentials	6
AssetManagement / AssetLeases	10
AssetManagement / AssetLicenses	14
AssetManagement / AssetMetadata	11
AssetManagement / AssetOwnershipTransfers	7
AssetManagement / AssetRegisters	13
AssetManagement / AssetReports	2
AssetManagement / AssetTypes	2
AssetManagement / Clubs	5
AssetManagement / Workflows	1
Auth / AbacAuthorizes	4
Auth / Accounts	13
Auth / CacheInvalidation	6
Auth / Files	14
Auth / Lookup	5
Auth / Notes	6
Auth / Tenants	8
Auth / UiPermissions	3
Auth / Users	6
Booking / BookingCatalog	6
Booking / BookingClass	11
Booking / BookingPricingChartDiscount	6
Booking / BookingTransferRequest	1
Booking / ClassManagement	2
Booking / ClassTerm	2
Booking / ProfileClassBooking	2
Booking / ProfileCourseBooking	3
Credential / Credentials	4
FieldManagement / EntityExtensions	18

6.5. Appendix B: Issues List

ID	Issue	Priority	Recommended Action
ISS-001	Source code for Finance, MemberProfile, Membership, Organisation, MobileApps, and Result modules is referenced but not present in the extracted source tree; only build outputs appear available.	High	Obtain full source and rescan.
ISS-002	Tenant isolation rules are not fully documented.	High	Validate tenant resolution, database routing, and cross-tenant authorization rules.
ISS-003	Production use of hash/encrypt/decrypt helper endpoints needs security review.	High	Confirm whether these endpoints are development-only or restrict/remove in production.
ISS-004	Business workflows for asset status transitions, leases, licenses, transfers, and approvals require stakeholder validation.	High	Workshop with product owners and support team.
ISS-005	Booking payment and finance flows are incomplete due to missing source modules.	High	Rescan full Finance and payment source.
ISS-006	API response schema and error code conventions require formal documentation.	Medium	Generate OpenAPI snapshot and define response standard.
ISS-007	Non-functional targets such as availability, throughput,	Medium	Confirm production SLA and compliance requirements.

	RTO, RPO, retention, and audit retention are TBD.		
ISS-008	Data model, ERD, stored procedures, and database schema were not available in this scan.	High	Obtain database schema/scripts and generate DR section.
ISS-009	Security model uses ABAC/UI permissions/custom middleware, but policy catalog is not documented.	High	Export and document policy names, action attributes, and field permissions.
ISS-010	Swagger may only show available compiled modules at runtime; static source scan cannot confirm missing module routes.	Medium	Run API locally with full source and export OpenAPI JSON.